



import requests

```
PUBLIC_URL = "https://identity-faucet-clubbed.ngrok-free.dev"
```

```
def ask(question, system="You are a helpful assistant."):
    response = requests.post(
        f"{PUBLIC_URL}/v1/chat/completions",
        headers={"Content-Type": "application/json"},
        json={
            "model": "phi-4-mini",
            "messages": [
                {"role": "system", "content": system},
                {"role": "user", "content": question}
            ],
            "max_tokens": 512
        }
    )
    return response.json()["choices"][0]["message"]["content"]
```

```
print(ask("What is the difference between 1NF and 2NF?"))
```

```
import time
```

Setup the LLM

```
ask("I will be providing you with an sql file. Use the file to answer the following questions regarding the schema.")
```

Keeps prompting the user to ask questions until they say exit

```
while True:
    print("What would you like to process from the database? Enter 'Exit' to stop the program.")
    user_data = input("Enter Here:")

    # Exit Condition
    exit = "Exit"
    if user_data == exit:
        break

    # The user can ask for data like "give me the list of all sandwiches in NJ" and we convert that to an SQL prompt and return the user the data they want using the database
```

```
ask(user_data)

print("Continuing Program...")
time.sleep(3)

print("You have successfully exited the program.")
```

How could I add a file to the LLM

You generally don't "add a file to the LLM" by sending the whole file inside your chat/completions request. The usual pattern is to either extract the file's text yourself and include that text in the prompt, or use an API feature built for file input instead of raw chat completions.^[1]

What to do instead

For your code, the simplest approach is:

1. Read the file in Python.
2. Extract the useful text from it.
3. Put that text into the messages content you send to the model.

For example, if the file is a text or SQL file, you can do something like this:

```
def ask(question, system="You are a helpful assistant.", file_path=None):
    file_text = ""
    if file_path:
        with open(file_path, "r", encoding="utf-8") as f:
            file_text = f.read()

    messages = [{"role": "system", "content": system}]
    if file_text:
        messages.append({
            "role": "user",
            "content": f"Here is the file content:\n\n{file_text}"
        })
    messages.append({"role": "user", "content": question})

    response = requests.post(
        f"{PUBLIC_URL}/v1/chat/completions",
        headers={"Content-Type": "application/json"},
        json={
            "model": "phi-4-mini",
            "messages": messages,
            "max_tokens": 512
        }
    )
    return response.json()["choices"][^1_0]["message"]["content"]
```

Better pattern for SQL files

If your file is an `.sql` schema file, the most practical workflow is to load it once and keep the schema text in memory for later questions. That avoids re-reading and re-sending the file every time through the loop.

```
with open("schema.sql", "r", encoding="utf-8") as f:
    schema_text = f.read()

def ask(question, system="You are a helpful assistant."):
    response = requests.post(
        f"{PUBLIC_URL}/v1/chat/completions",
        headers={"Content-Type": "application/json"},
        json={
            "model": "phi-4-mini",
            "messages": [
                {"role": "system", "content": system},
                {"role": "user", "content": f"Use this schema:\n\n{schema_text}\n\nQ: {question}"},
            ],
            "max_tokens": 512
        }
    )
    return response.json()["choices"][^1_0]["message"]["content"]
```

When files are too large

If the file is big, don't paste all of it into the prompt. Instead, split it into chunks, summarize it, or use retrieval/search over the file contents before sending only the relevant parts to the model. Chat completions are not meant for file storage, and for file-upload-style workflows the docs point to newer APIs designed for that purpose.^[1]

Important fix in your loop

Right now you call:

```
ask("I will be providing you with an sql file. Use the file to answer the following q")
```

but that doesn't actually send a file. You need to read the file in Python and attach its contents to the prompt, or use a file-aware API endpoint.^[1]

Would you like me to rewrite your script so it loads an `.sql` file once and answers questions from it?

✱

1. <https://llms-developer-docs.netlify.app/platform/overview/upload-files/>
2. <https://www.emergentmind.com/topics/llm-generated-context-files>
3. <https://www.speakeasy.com/api-design/file-uploads>
4. <https://github.com/openai/openai-agents-python/issues/893>

5. https://www.reddit.com/r/learnmachinelearning/comments/1fitoin/how_can_i_provide_a_large_amount_of_context_to_an/
6. <https://dev.to/es404020/uploading-files-to-openai-passing-the-baton-1f96>
7. <https://community.openai.com/t/send-files-to-completion-api/575385>
8. <https://stackoverflow.com/questions/22567306/how-to-upload-file-with-python-requests>
9. <https://github.com/simonw/llm/issues/523>
10. <https://apxml.com/courses/python-llm-workflows/chapter-3-interacting-llm-apis-python/api-requests-with-requests>
11. https://www.reddit.com/r/LLMDevs/comments/1ieddgs/attach_files_in_api_request/
12. <https://community.make.com/t/openai-upload-a-file-openai-and-have-the-next-step-create-a-completion-openai-to-access-the-file/20346>
13. https://www.youtube.com/watch?v=6eZ91_QdHKU
14. <https://docs.x.ai/developers/rest-api-reference/files/upload>
15. <https://community.openai.com/t/open-ai-api-file-upload-limitations-on-chat-completions-api/1372667>